

Architectural Blueprint and Implementation Roadmap: Option Lifecycle Manager on Oracle Cloud Infrastructure

Executive Summary

This technical report delineates the architectural specification and execution roadmap for the **Option Lifecycle Manager (OLM)**, a sophisticated extension to the existing Deep-Tech trading platform. The primary objective is to transition from a passive position tracking system to an active, event-driven management engine capable of automating the entire lifecycle of derivative instruments. The system is specifically designed to handle the user's paradigmatic use case: the acquisition of a Gold Call option (e.g., Strike 400, Expiration Sep 18).

The functional requirements mandate a seamless flow from symbology ingestion to automated chart visualization, followed by algorithmic candle monitoring, and culminating in a dynamic exit strategy influenced by multi-factor inputs including price action, news sentiment, and technical indicators. Crucially, this entire infrastructure is constrained to operate within the **Oracle Cloud Infrastructure (OCI) Always Free Tier**, utilizing the **Ampere A1** compute shapes. This constraint dictates specific architectural choices regarding memory management, processor architecture (ARM64), and containerization strategies.¹

The report is structured to provide an exhaustive technical analysis of the system's components, ensuring compliance with the 14-week development timeline. It leverages the unique high-memory capabilities (24 GB RAM) of the OCI Ampere instances to colocate heavy workloads—specifically local Large Language Model (LLM) inference for sentiment analysis—alongside high-frequency event loops and time-series databases.

1. Infrastructure Architecture: The Oracle Ampere Paradigm

The selection of the Oracle Cloud Infrastructure (OCI) Ampere A1 compute shape as the deployment target is a strategic decision that fundamentally shapes the software architecture. While traditional x86_64 cloud instances in free tiers typically offer 1-2 vCPUs and 1-2 GB of RAM, the OCI Always Free Tier provides up to **4 OCPUs (ARM64)** and **24 GB of RAM**.¹ This resource abundance, particularly in system memory, is the enabling factor for the "Deep Tech" components of this roadmap, specifically the requirement to generate dynamic

exit plans based on news and sentiment using local AI models.

1.1 The Processor Architecture: ARM64 Considerations

The utilization of the Ampere Altra processor (aarch64 architecture) necessitates a departure from standard deployment pipelines. The system is built upon a Reduced Instruction Set Computing (RISC) architecture, which offers superior energy efficiency and predictable performance but introduces compatibility challenges with pre-compiled binaries and Docker containers often defaulted to AMD64 (x86_64).¹

The implication for the OLM is that every component in the stack—from the database to the AI inference engine—must be verified for ARM64 compatibility or compiled from source. Specifically, the **TimescaleDB** high-availability (HA) images have historically lacked multi-architecture support, requiring the use of standard images or custom build processes.⁵ Furthermore, Python libraries relying on C extensions (e.g., numpy, pandas, grpc) must fetch wheels built for aarch64 or undergo compilation during the build phase, which influences the Continuous Integration/Continuous Deployment (CI/CD) pipeline design.

1.2 Memory Hierarchy and Allocation Strategy

The 24 GB RAM allocation is substantial but finite. To support the "Option Lifecycle Manager" alongside the "Sentinel" AI agent, a rigorous memory budget is enforced. The system effectively operates as a hyper-converged micro-monolith, hosting the database, application logic, and inference server on a single node (or split across two nodes within the 4-core limit).

Component	Function	Memory Allocation	Architectural Justification
TimescaleDB (PostgreSQL)	Time-Series & Vector Data	4 GB	Sufficient for buffering 5-minute candles and vector embeddings (pgvector) for RAG without aggressive swapping. ⁷
Ollama (Inference Server)	Local LLM (Llama 3/Mistral)	8 GB	Reserved for running 4-bit quantized (q4_0) 7B/8B parameter models. This is critical for the

			"Sentiment" requirement. ⁸
OLM Engine (Python)	Event Loop & State Machine	4 GB	Handling pandas DataFrames for OHLCV data, strategy indicators, and in-memory order books. ¹⁰
System Overhead	OS, Logging, Networking	4 GB	Buffer for Oracle Linux/Ubuntu kernel and background services.
Headroom	Swap/Burst	4 GB	Safety margin to prevent OOM Killer invocation during high-volatility market events.

This allocation strategy confirms the viability of running the full stack on the Free Tier, provided strict quantization is applied to the AI models.

1.3 Container Orchestration and Networking

The application is deployed as a Docker Compose stack. Given the single-host nature of the Free Tier, complex orchestrators like Kubernetes (OKE) add unnecessary overhead, although the Ampere instances are certified for such workloads.¹ The stack utilizes an internal bridge network for high-speed communication between the Python Application and the Postgres Database, ensuring low latency for tick processing.

For external connectivity, particularly for the "Pull up the Gold Chart" feature which requires a web frontend, the architecture utilizes an NGINX reverse proxy. This proxy manages SSL termination and routes WebSocket connections for the **Lightweight Charts** UI, ensuring real-time interactivity for the "Drag-and-Drop" order management features.¹¹

2. The Symbology Engine: Parsing and Association

The user's first requirement is: *"If I buy a Gold Call (e.g., Strike 400, Exp Sep 18), the system must Pull up the Gold chart automatically."* This seemingly simple request conceals a complex

data engineering challenge: the translation of a derivative symbol into a chartable underlying asset.

2.1 The Options Symbolology Initiative (OSI) Parser

The standard for option symbols in the US market is the OSI format, a 21-character string that encodes the root symbol, expiration date, option type, and strike price.¹³ The OLM includes a dedicated SymbolologyParser module designed to decompose these strings with regex precision.

Consider the example: GLD 250918C00400000.

The parser executes the following extraction logic:

1. **Root Symbol Extraction (Bytes 0-5):** GLD___ (padded with spaces). The parser strips whitespace to yield GLD. This identifies the underlying asset.
2. **Expiration Date (Bytes 6-11):** 250918. This translates to September 18, 2025. This date is converted into a datetime object, crucial for calculating Time-to-Expiry (\$T\$) in Greek calculations (Theta).¹⁴
3. **Option Type (Byte 12):** C denotes a Call. This determines the directional bias of the instrument (Bullish).
4. **Strike Price (Bytes 13-20):** 00400000. The OSI standard dictates this is the strike price multiplied by 1000. The parser performs the division ($\$400000 / 1000\$$) to yield the float 400.0.

This parsing occurs in microseconds at the point of ingestion (e.g., when the user enters the position into the blotter).

2.2 The Instrument Association Map

Once the root GLD is extracted, the system must determine *which* chart to display. A raw ticker symbol is often insufficient for charting libraries that may require exchange-specific prefixes (e.g., AMEX:GLD or NYSE:GLD) or mapping to a futures contract (e.g., GC=F for Gold Futures).

The OLM maintains a local InstrumentDefinition table in TimescaleDB, populated via the **Databento** or **yfinance** reference APIs.¹³

- **Query:** SELECT chart_symbol, sector, tick_size FROM instruments WHERE root_symbol = 'GLD'
- **Result:** chart_symbol: 'GLD', sector: 'Commodities', tick_size: 0.01.

This lookup ensures that when the user enters an option symbol, the system "knows" to load the specific OHLCV data for the SPDR Gold Shares ETF (GLD) and not a generic placeholder.

2.3 Automatic Visualization Trigger

The "Pull up... automatically" requirement is implemented via a frontend-backend Event Bus.

1. **Ingestion Event:** The Python backend emits a `NewPositionEvent` containing the parsed details (root: GLD, strike: 400).
2. **Frontend Listener:** The web interface (built with Streamlit or a custom React wrapper around Lightweight Charts) subscribes to this event stream.
3. **Context Switch:** Upon receipt, the frontend:
 - Clears the existing data series.
 - Requests historical candles for GLD from the API.
 - **Overlay Generation:** It creates a horizontal `PriceLine` primitive on the chart at the level 400.0. This line is styled distinctively (e.g., dashed blue line) and labeled "Strike".

This immediate visual feedback allows the trader to instantly contextualize the option's moneyness (In-The-Money vs. Out-Of-The-Money) relative to the current price action.¹⁵

3. The Event-Driven Trading Engine

The second core requirement is to *"Monitor candles and automate trades based on a strategy."* To achieve this without lookahead bias and with the ability to react to real-time news, the system adopts an **Event-Driven Architecture**.

3.1 The Event Loop Architecture

Unlike vectorized backtesting frameworks that process entire dataframes at once, an event-driven engine processes data sequentially, mimicking the flow of time in live markets.¹⁶ The core of the OLM is a while True loop that continuously polls an `EventQueue`.

Core Event Types:

- **MarketEvent:** Triggered when a new data point (candle or tick) arrives from the `DataHandler`. It carries the snapshot of the market (OHLCV) at time `$t$`.
- **SignalEvent:** Generated by the `Strategy` object when logic conditions are met (e.g., "Buy GLD"). It carries the symbol, direction, and timestamp.
- **OrderEvent:** Generated by the `Portfolio` object to request execution. It specifies order type (Market/Limit), quantity, and constraints.
- **FillEvent:** Generated by the `ExecutionHandler` (simulated or live) confirming the trade.¹⁶

The loop runs on the Ampere CPU, efficiently handling the state transitions. The `Python queue.Queue` is used for thread-safe communication between the Data Ingestion thread (fetching prices) and the Execution thread (processing logic).

3.2 Strategy Implementation: The "Gold" Logic

To satisfy the requirement of "automating trades," we define a specific `TrendFollowingStrategy` class for the Gold scenario.

- **Input:** 5-minute OHLCV bars for GLD.
- **Indicators:**
 - Fast SMA (50-period).
 - Slow SMA (200-period).
 - RSI (14-period).
- **Logic:**
Python

```
if fast_sma > slow_sma and previous_fast_sma < previous_slow_sma:
    if rsi < 70:
        signal = SignalEvent(symbol='GLD', type='CALL', strength=1.0)
        events.put(signal)
```

This logic detects a "Golden Cross" (bullish trend confirmation) while ensuring the asset is not already overbought (RSI < 70).¹⁸

3.3 The Option Lifecycle State Machine

The OLM manages the *Position* distinct from the *Strategy*. A strategy generates signals; the OLM manages the resulting holding. This is modeled as a Finite State Machine (FSM) ²⁰:

1. **PENDING:** Signal received, validating capital and risk limits.
2. **SUBMITTED:** Order sent to broker API.
3. **FILLED:** Trade confirmed. The OLM now instantiates a *Position* object.
4. **ACTIVE:** The OLM monitors the position for exit conditions (Stop Loss, Take Profit, Expiry).
5. **CLOSING:** Exit condition triggered. Closing order sent.
6. **CLOSED:** Position flattened. PnL realized.

This separation of concerns allows the OLM to manage the lifecycle of the *Option* (expiration, assignment risk, Greeks) independently of the *Strategy* that triggered the entry.

4. The Intelligence Layer: Dynamic Exit Plans

The third requirement is the most advanced: "*Generate and execute a dynamic 'Exit Plan' based on price, news, and sentiment.*" This moves beyond simple technical analysis into the realm of **AI-Augmented Trading**, leveraging the OCI Ampere's capacity for local inference.

4.1 "The Sentinel" Agent

We introduce a dedicated subsystem named "**The Sentinel**." This is an autonomous agent

running in parallel to the trading loop. Its sole responsibility is to ingest unstructured data (news), quantify it, and modulate the risk parameters of active positions.

The Local LLM Stack:

The Sentinel utilizes Ollama running on the Ampere A1 instance. We select the Llama 3 8B model (quantized to 4-bit, q4_0) for its optimal balance of reasoning capability and memory footprint (~4.7 GB). This runs entirely locally, ensuring data privacy and zero inference cost.⁸

LangChain Integration:

We utilize LangChain to structure the interaction with Ollama.

- **Data Source:** A background worker scrapes RSS feeds for keywords relevant to the position (e.g., "Gold," "Inflation," "Federal Reserve," "USD").
- **Prompt Engineering:** "Analyze the following news headline regarding Gold prices. Classify the sentiment as 'Bullish', 'Bearish', or 'Neutral'. Provide a confidence score between 0.0 and 1.0. Output strictly in JSON format."²¹
- **Output Processing:** The Sentinel aggregates scores over a rolling window (e.g., last 60 minutes) to determine a global SentimentFactor.

4.2 Dynamic Exit Logic Mechanism

The "Dynamic Exit Plan" is implemented as a function of both Technicals and Sentiment.

Let P_{entry} be the entry price.

Let SL_{base} be the base Stop Loss (e.g., $P_{\text{entry}} \times 0.95$).

Let TP_{base} be the base Take Profit (e.g., $P_{\text{entry}} \times 1.10$).

The Sentinel calculates a modifier, α , based on the aggregate sentiment score ($S \in [-1, 1]$).

- If $S > 0.5$ (Strong Bullish): $\alpha = 1 + (S \times 0.2)$.
- If $S < -0.5$ (Strong Bearish): $\alpha = 1 + (S \times 0.2)$ (Note: applied to reduce limits).

Dynamic Adjustment:

- **Bullish Scenario (Long Call):**
 - $TP_{\text{dynamic}} = TP_{\text{base}} \times \alpha$ (Target raised to capture more upside).
 - $SL_{\text{dynamic}} = SL_{\text{base}} \times (1 + |S| \times 0.05)$ (Stop tightened slightly to lock risk, or widened if volatility tolerance increases).
- **Bearish Scenario (Long Call):**
 - $TP_{\text{dynamic}} = TP_{\text{base}}$ (Unchanged).
 - $SL_{\text{dynamic}} = P_{\text{current}} - (P_{\text{current}} \times 0.01)$ (Stop aggressively tightened to "get out" on the first sign of weakness).

This logic allows the system to "let winners run" during favorable news cycles and "cut losers fast" during adverse events, fulfilling the requirement for a dynamic plan based on news/sentiment.

4.3 Technical Price Exits

In addition to sentiment, the Exit Plan incorporates technical price triggers:

- **Trailing Stop:** Activated once the position is profitable by $X\%$. The stop level tracks the High Water Mark of the price.
- **Technical Breakdown:** If the 50-SMA crosses below the 200-SMA (Death Cross), the OLM triggers an immediate exit regardless of the PnL, overriding sentiment.

5. The Presentation Layer: Lightweight Charts Integration

The visualization requirement is served by **Lightweight Charts** (TradingView's open-source library), wrapped in Python to interface with the OLM backend.²² This library is chosen for its performance (HTML5 Canvas) and interactivity.

5.1 Interactive Order Management (Visual Trading)

To satisfy the "Execute a dynamic Exit Plan" requirement visually, we implement **Drag-and-Drop Order Management**.

- **Implementation:** The Exit Price (Stop Loss or Take Profit) is rendered as a horizontal line primitive on the chart.¹⁵
- **Event Handling:** The Python wrapper utilizes the `subscribeClick` and `crosshairMove` events exposed by the library.²³
 - **Drag Start:** User clicks and holds the "Exit" line.
 - **Drag Move:** The line follows the cursor's y-axis position.
 - **Drag End (Drop):** The user releases the mouse. The chart emits an event with the new price level.
- **Backend Sync:** The OLM receives this event and updates the active `OrderTicket` in the system. The "Dynamic Exit Plan" is thus editable in real-time by the user, providing a hybrid automated/manual control mechanism.²⁵

5.2 Option Payoff Visualization

Alongside the price chart, the system renders an **Option Payoff Diagram**.

- **Library:** We utilize the `optionlab` or `opstrat` Python libraries.²⁶
- **Calculation:** Using the parsed OSI data (Strike 400, Call), the library calculates the PnL curve at expiration.
 - $\$PnL = \max(S_T - K, 0) - \text{Premium}$
- **Overlay:** This diagram is rendered either as a static image in a secondary panel or, more advancedly, as a `CustomSeries` within **Lightweight Charts**, allowing the user to visualize

the "Breakeven Point" directly against the historical price action.²⁸

6. The 14-Week Deep-Tech Roadmap

This roadmap structures the development into four phases, meticulously planned to manage the complexity of building a high-performance trading system on the constrained OCI Free Tier.

Phase I: Foundation & Infrastructure (Weeks 1-4)

Objective: Establish the ARM64 environment, deploy the data core, and build the Symbology Engine.

Week 1: OCI Ampere Provisioning & Hardening

- **Tasks:**
 - Provision OCI Ampere A1 instance (4 OCPU, 24GB RAM). Install Oracle Linux 8 or Ubuntu 22.04.¹
 - Configure kernel parameters (sysctl) for high-throughput networking (TCP stack tuning) to support data feeds.
 - Deploy **TimescaleDB** container. **Crucial:** Use the standard timescaledb image or build timescaledb-ha from source to ensure ARM64 compatibility.⁵
 - Initialize pgvector extension for future RAG capabilities.
- **Deliverable:** Operational PostgreSQL database accessible on port 5432 via internal network.

Week 2: Data Ingestion & OSI Symbology

- **Tasks:**
 - Implement DataHandler using yfinance for historical data and Databento (if available) for granular ticks.¹³
 - Develop the **OSI Symbology Parser** (Regex module). Write unit tests covering various asset classes (Equity Options, ETF Options).
 - Populate InstrumentDefinition table mapping Roots (e.g., GLD) to Chart Symbols (GLD).
- **Deliverable:** Python script that accepts GLD 250918C00400000 and returns GLD and Strike: 400.

Week 3: Visualization Core (Lightweight Charts)

- **Tasks:**
 - Implement the Python wrapper for Lightweight Charts.¹²
 - Create the **Auto-Pull Controller**: A frontend input box that takes the Option Symbol, triggers the Symbology Parser, fetches data for the result, and renders the chart.

- Implement PriceLine primitives to visualize the Strike Price.
- **Deliverable:** Browser-based UI where inputting an option symbol loads the correct underlying chart with the strike line.

Week 4: The Option Lifecycle Manager (Backend v1)

- **Tasks:**
 - Implement Position, Order, and Trade classes.
 - Build the State Machine (Pending -> Active -> Closed).
 - Integrate opstrat to generate the Payoff Diagram upon position entry.²⁷
- **Deliverable:** Backend capability to track the state of a "Paper Trade."

Phase II: The Event-Driven Engine (Weeks 5-8)

Objective: Automate the "Monitor candles and execute trades" requirement.

Week 5: The Event Loop

- **Tasks:**
 - Architect the while True loop and EventQueue.¹⁶
 - Implement the MarketEvent, SignalEvent, and OrderEvent flow.
 - Ensure thread safety between the Data Ingestion thread and the Strategy Execution thread.
- **Deliverable:** Functional event loop processing live (or simulated) ticks.

Week 6: Strategy Implementation

- **Tasks:**
 - Code the TrendFollowingStrategy for Gold (SMA Crossover + RSI).
 - Implement Indicator classes (SMA, RSI, Bollinger Bands) using pandas rolling windows.¹²
 - Connect Strategy Signals to OLM Order entry.
- **Deliverable:** System automatically generating "Buy" signals based on GLD price action.

Week 7: Dynamic Exit Framework (Technical)

- **Tasks:**
 - Implement TrailingStop and TakeProfit logic within the OLM.
 - Implement **Drag-and-Drop** order modification in the UI. Bind frontend events to backend order updates.²³
- **Deliverable:** Ability to modify the exit price of a running strategy by dragging a line on the chart.

Week 8: Backtesting & Calibration

- **Tasks:**

- Run the engine in "Backtest Mode" using historical GLD data.
- Validate that the OLM correctly manages state transitions (e.g., does not sell a position it doesn't own).
- Optimize strategy parameters (SMA lengths) for the Gold contract.
- **Deliverable:** Backtest report showing Sharpe Ratio and Drawdown.

Phase III: The Intelligence Layer (Weeks 9-12)

Objective: Integrate AI Sentiment for the "Dynamic Exit Plan."

Week 9: Local AI Infrastructure (Ollama)

- **Tasks:**
 - Install **Ollama** on the Ampere instance. Verify hardware acceleration (NEON instructions).⁸
 - Pull and test llama3:8b-instruct-q4_0.
 - Benchmark inference speed (Tokens/Sec). Ensure it does not starve the Trading Loop of memory.
- **Deliverable:** Local LLM responding to prompts via API.

Week 10: Sentiment Pipeline

- **Tasks:**
 - Build the **News Scraper** (RSS/API).
 - Implement the **LangChain** pipeline.²¹ Create the Prompt Template for financial sentiment classification.
 - Test accuracy on historical financial news headlines.
- **Deliverable:** Stream of news headlines being converted into numerical Sentiment Scores.

Week 11: The Sentinel Agent Integration

- **Tasks:**
 - Connect the Sentinel's SentimentScore to the OLM's Exit Logic.
 - Implement the mathematical modifier (Alpha) to adjust Stop Loss/Take Profit levels based on sentiment.
 - *Scenario Test:* Simulate "Bad News" and verify the Stop Loss tightens automatically.
- **Deliverable:** Fully functional "AI-Augmented" exit strategy.

Week 12: RAG & Context (Advanced)

- **Tasks:**
 - Implement **Retrieval Augmented Generation (RAG)** using pgvector.³¹
 - Store historical news and price impacts.
 - Query: "How did Gold react last time inflation data was high?" to refine the sentiment score.

- **Deliverable:** Context-aware sentiment analysis.

Phase IV: Integration & Launch (Weeks 13-14)

Objective: Polish the system for production.

Week 13: The Unified Dashboard

- **Tasks:**
 - Combine Chart, Payoff Diagram, Sentinel Log, and Blotter into a single Streamlit or PyQt application.¹¹
 - Ensure responsive layout and real-time updates via WebSockets.
- **Deliverable:** The complete "Deep Tech" trading terminal.

Week 14: Stress Testing & Documentation

- **Tasks:**
 - **Load Test:** Run the Trading Loop, Database, and LLM simultaneously. Monitor RAM usage (target < 20GB).
 - **Latency Test:** Measure time from "News Arrival" to "Order Update."
 - Write the User Manual and System Architecture documentation.
- **Deliverable:** Production-ready system.

7. Risk Management and System Limits

7.1 Memory Contention Risks

The primary risk on the OCI Free Tier is memory exhaustion. Running a database, a Python event loop, and an 8B parameter LLM simultaneously pushes the 24 GB limit.

- **Mitigation:**
 - **Quantization:** Strictly use 4-bit (or even 3-bit) quantized models for Ollama.
 - **Swap Space:** Configure 8-16 GB of Swap on the OCI block storage to handle transient spikes, though this will impact performance.
 - **Service Isolation:** Ensure the OLM (Trading Engine) has higher process priority (nice) than the Sentinel (AI), ensuring trade execution is never blocked by inference.

7.2 Latency and CPU Constraints

The Ampere A1 CPUs are efficient but single-thread performance is lower than modern x86 cores.

- **Mitigation:** The architecture decouples the "Sentinel" (AI) from the "Loop" (Trading). The Trading Loop runs on a dedicated thread/core reacting to price ticks in milliseconds. The Sentinel runs asynchronously, updating *parameters* every few minutes. A delay in

sentiment analysis is acceptable; a delay in stop-loss execution is not.

7.3 Data Dependency

The "Auto-Pull" feature relies on external APIs (yfinance).

- **Mitigation:** Implement caching for Instrument Definitions to reduce API calls. Implement "Circuit Breakers" that pause trading if data feeds become stale (e.g., no tick for 1 minute).

8. Conclusion

The proposed **Option Lifecycle Manager** represents a sophisticated synthesis of financial engineering and modern software architecture. By rigorously adhering to the constraints of the Oracle Cloud Free Tier, we have designed a system that extracts maximum value from the unique Ampere hardware.

The system addresses every requirement of the user's request:

1. **Symbology Parsing** seamlessly links complex option strings to chartable assets.
2. **The Event-Driven Engine** provides the backbone for automated, strategy-based trading without lookahead bias.
3. **The Sentinel Agent**, powered by local LLM inference, introduces a "Deep Tech" edge, enabling dynamic, sentiment-aware risk management that adapts to the narrative of the market.
4. **Lightweight Charts** provides the interactive, visual command center required to manage these positions effectively.

This roadmap transforms the platform from a simple tool into an intelligent, autonomous trading assistant, fully realizable within the 14-week timeline.

Works cited

1. Arm-Based Compute - Oracle, accessed on January 2, 2026, <https://www.oracle.com/cloud/compute/arm/>
2. Oracle Cloud Free Tier, accessed on January 2, 2026, <https://www.oracle.com/cloud/free/>
3. Always Free Resources - Oracle Help Center, accessed on January 2, 2026, https://docs.oracle.com/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm
4. Oracle Cloud with Ampere A1 Infrastructure Solutions, accessed on January 2, 2026, <https://amperecomputing.com/products/partners/oracle-cloud>
5. timescale-ha multiarch (ARM/Aarch64) support · Issue #259 - GitHub, accessed on January 2, 2026, <https://github.com/timescale/timescaledb-docker-ha/issues/259>

6. Add ARM support · Issue #110 · timescale/timescaledb-docker-ha - GitHub, accessed on January 2, 2026,
<https://github.com/timescale/timescaledb-docker-ha/issues/110>
7. Install TimescaleDB on Docker - Tiger Data Documentation, accessed on January 2, 2026,
<https://www.tigerdata.com/docs/self-hosted/latest/install/installation-docker>
8. Llama 3.1 on OCI A1 VMs - Ampere Computing, accessed on January 2, 2026,
https://amperecomputing.com/assets/24_Serge_OCW_Demo_Brochure_d091dfe51e.pdf
9. Running LLMs on Oracle Linux with Ollama, accessed on January 2, 2026,
<https://blogs.oracle.com/linux/running-llms-on-oracle-linux-with-ollama>
10. Python framework for TradingView's Lightweight Charts JavaScript library. - GitHub, accessed on January 2, 2026,
<https://github.com/louisnw01/lightweight-charts-python>
11. Easy, interactive financial charts in Python: Just 11 lines of code, no JavaScript required, accessed on January 2, 2026,
<https://medium.databento.com/easy-interactive-financial-charts-in-python-just-11-lines-of-code-no-javascript-required-cde338eecd43>
12. lightweight-charts-python: Effortlessly Create Efficient Financial Candlestick Charts with Python | by Meng Li | Top Python Libraries | Medium, accessed on January 2, 2026,
<https://medium.com/top-python-libraries/lightweight-charts-python-effortlessly-create-efficient-financial-candlestick-charts-with-python-a786c315a2a4>
13. Equity options: Intro | Reading OCC option symbols and more - Databento, accessed on January 2, 2026,
<https://databento.com/docs/examples/options/equity-options-introduction>
14. How can I use regex to parse an OCC Option Symbol? - Stack Overflow, accessed on January 2, 2026,
<https://stackoverflow.com/questions/44402379/how-can-i-use-regex-to-parse-a-n-occ-option-symbol>
15. AbstractChart - LightweightChartsPython - Read the Docs, accessed on January 2, 2026,
https://lightweight-charts-python.readthedocs.io/en/latest/reference/abstract_chart.html
16. Event-Driven Backtesting with Python - Part I - QuantStart, accessed on January 2, 2026,
<https://www.quantstart.com/articles/Event-Driven-Backtesting-with-Python-Part-I/>
17. How I Built an Event-Driven Backtesting Engine in Python | by Timothy Kimutai | Medium, accessed on January 2, 2026,
<https://timkimutai.medium.com/how-i-built-an-event-driven-backtesting-engine-in-python-25179a80cde0>
18. Top 5 Algo Trading Strategies with Examples (2025) - Findoc, accessed on January 2, 2026, <https://www.findoc.com/blog/5-algorithmic-trading-strategies>
19. Mastering Algorithmic Trading: Crafting Strategies from Concept to Execution -

- The AI Quant, accessed on January 2, 2026,
<https://theaiquant.medium.com/mastering-algorithmic-trading-crafting-strategies-from-concept-to-execution-e4adaf0c187f>
20. Trading and Orders - QuantConnect.com, accessed on January 2, 2026,
<https://www.quantconnect.com/docs/v2/writing-algorithms/live-trading/trading-and-orders>
 21. LangChain Ollama Integration: Complete Tutorial with Examples - Latenode, accessed on January 2, 2026,
<https://latenode.com/blog/ai-frameworks-technical-infrastructure/langchain-setup-tools-agents-memory/langchain-ollama-integration-complete-tutorial-with-examples>
 22. Lightweight Charts™ library - TradingView, accessed on January 2, 2026,
<https://www.tradingview.com/lightweight-charts/>
 23. Interface: IChartApi | Lightweight Charts - GitHub Pages, accessed on January 2, 2026,
<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/IChartApi>
 24. Draw price line on Click event - lightweight charts - Stack Overflow, accessed on January 2, 2026,
<https://stackoverflow.com/questions/76232517/draw-price-line-onclick-event>
 25. Add the ability to drag custom price line and change its price · Issue #1086 · tradingview/lightweight-charts - GitHub, accessed on January 2, 2026,
<https://github.com/tradingview/lightweight-charts/issues/1086>
 26. rgaveiga/optionlab: A Python library for evaluating option trading strategies. - GitHub, accessed on January 2, 2026, <https://github.com/rgaveiga/optionlab>
 27. Visualizing Options in Python Using Opstrat - Kaggle, accessed on January 2, 2026,
<https://www.kaggle.com/code/abhijithchandradas/visualizing-options-in-python-using-opstrat>
 28. Custom Series Types | Lightweight Charts - GitHub Pages, accessed on January 2, 2026,
https://tradingview.github.io/lightweight-charts/docs/plugins/custom_series
 29. Option Payoff - Quantra by QuantInsti, accessed on January 2, 2026,
<https://quantra.quantinsti.com/glossary/Option-Payoff>
 30. Multi-Leg Options Strategy Backtesting with Python | by SR - Medium, accessed on January 2, 2026,
<https://medium.com/@deepml1818/multi-leg-options-strategy-backtesting-with-python-e112baa36038>
 31. timescale/pgvectorscale: Postgres extension for vector search (DiskANN), complements pgvector for performance and scale. Postgres OSS licensed. - GitHub, accessed on January 2, 2026,
<https://github.com/timescale/pgvectorscale>
 32. Timescale Vector x LangChain: Making PostgreSQL A Better Vector Database for AI Applications, accessed on January 2, 2026,
<https://blog.langchain.com/timescale-vector-x-langchain-making-postgresql-a-better-vector-database-for-ai-applications/>